



"STRINGS"

Basic String Operations

- **Many types of programs perform operations on strings**
- **In Python, many tools for examining and manipulating strings**
 - Strings are sequences, so many of the tools that work with sequences work with strings

Accessing the Individual Characters in a String

- **To access an individual character in a string:**
 1. **Use a `for` loop**
 - Format: `for character in string:`
 - Useful when need to iterate over the whole string, such as to count the occurrences of a specific character
 2. Use indexing

Figure 8-1 Iterating over the string 'Juliet'

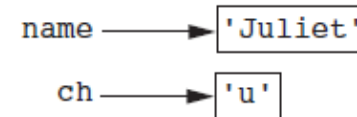
1st Iteration

```
for ch in name:  
    print(ch)
```



2nd Iteration

```
for ch in name:  
    print(ch)
```



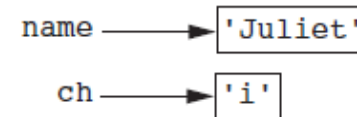
3rd Iteration

```
for ch in name:  
    print(ch)
```



4th Iteration

```
for ch in name:  
    print(ch)
```



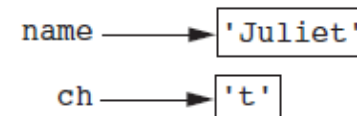
5th Iteration

```
for ch in name:  
    print(ch)
```



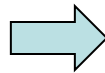
6th Iteration

```
for ch in name:  
    print(ch)
```



Accessing the Individual Characters Using Loop

```
1 name = 'Juliet'  
2 for ch in name:  
3     print(ch)
```



J
u
l
i
e
t

Program 8-1 (count_Ts.py)

```
1 # This program counts the number of times
2 # the letter T (uppercase or lowercase)
3 # appears in a string.
4
5 def main():
6     # Create a variable to use to hold the count.
7     # The variable must start with 0.
8     count = 0
9
10    # Get a string from the user.
11    my_string = input('Enter a sentence: ')
12
13    # Count the Ts.
14    for ch in my_string:
15        if ch == 'T' or ch == 't': ↔ if ch.capitalize() == 'T':        ↔ if ch.upper() == 'T':
16            count += 1
17
18    # Print the result.
19    print('The letter T appears', count, 'times.')
20
21 # Call the main function.
22 main()
```

Program Output (with input shown in bold)

Enter a sentence: **Today we sold twenty-two toys** ↵
The letter T appears 5 times.



count_Ts.py

Accessing the Individual Characters in a String

- **To access an individual character in a string:**
 1. Use a for loop
 - Format: `for character in string:`
 2. **Use indexing**
 - Each character has an index specifying its position in the string, starting at 0
 - Format: `character = my_string[i]`

Accessing the Individual Characters in a String (cont'd.)

Figure 8-2 String indexes

'R o s e s a r e r e d'

↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑

0 1 2 3 4 5 6 7 8 9 10 11 12

You can use an index to retrieve a copy of an individual character in a string, as shown here:

```
my_string = 'Roses are red'
```

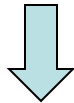
```
ch = my_string[6]
```


Accessing the Individual Characters in a String (cont'd.)

Another example:

```
my_string = 'Roses are red'
```

```
print(my_string[0], my_string[6], my_string[10])
```



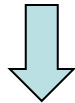
R a r

Accessing the Individual Characters in a String (cont'd.)

Another example:

```
my_string = 'Roses are red'
```

```
print(my_string[-1], my_string[-2], my_string[-13])
```



d e R

Accessing the Individual Characters in a String (cont'd.)

- **IndexError exception will occur if:**
 - You try to use an index that is out of range for the string

```
city = 'Boston'  
print(city[6])
```



Accessing the Individual Characters in a String (cont'd.)

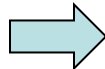
- **IndexError exception will occur if:**
 - You try to use an index that is out of range for the string
 - Likely to happen when loop iterates beyond the end of the string

```
city = 'Boston'  
index = 0  
while index < 7:  
    print(city[index])  
    index += 1
```

Accessing the Individual Characters in a String (cont'd.)

- **`len(string)` function can be used to obtain the length of a string**
 - 🍌 Useful to prevent loops from iterating beyond the end of a string

```
city = 'Boston'  
size = len(city)
```



```
city = 'Boston'  
  
index = 0  
  
while index < len(city):  
    print(city[index])  
    index += 1
```

Accessing the Individual Characters in a String (cont'd.)

- `len(string)` function can be used to obtain the length of a string
 - 🍌 Useful to prevent loops from iterating beyond the end of a string

```
city = 'Boston'
```

Even better 🤗 ➡

```
for index in range(len(city)):
    print(city[index])
```

String Concatenation

- **Concatenation: appending one string to the end of another string**
 - Use the + operator to produce a string that is a combination of its operands
 - The augmented assignment operator += can also be used to concatenate strings
 - The operand on the left side of the += operator must be an existing variable; otherwise, an exception is raised

Program 8-2 (concatenate.py.py)

```
1 # This program concatenates strings.
```

```
2
```

```
3 def main():
```

```
4     name = 'Carmen'
```

```
5     print('The name is', name)
```

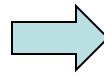
```
6     name = name + ' Brown'
```

```
7     print('Now the name is', name)
```

```
8
```

```
9 # Call the main function.
```

```
10 main()
```



Program Output (with input shown in bold)

The name is Carmen

Now the name is Carmen Brown.



concatenate.py

Strings Are Immutable (cont'd.)

Figure 8-4 The string 'Carmen' assigned to `name`

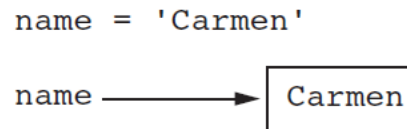
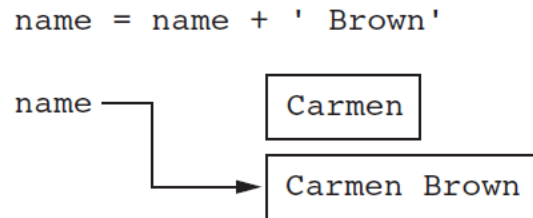


Figure 8-5 The string 'Carmen Brown' assigned to `name`



String Concatenation

- **Example i:**

```
1 >>> message = 'Hello ' + 'world' ↵  
2 >>> print(message) ↵  
3 Hello world  
4 >>>
```

- **Example ii:**

```
1 >>> first_name = 'Emily' ↵  
2 >>> last_name = 'Yeager' ↵  
3 >>> full_name = first_name + ' ' + last_name ↵  
4 >>> print(full_name) ↵  
5 Emily Yeager  
6 >>>
```

String Concatenation

- **Example iii:**

```
1 >>> letters = 'abc' ↵
2 >>> letters += 'def' ↵
3 >>> print(letters) ↵
4 abcdef
5 >>>
```

- **Example iv:**

<pre>>>> name = 'Kelly' ↵ >>> name += ' ' ↵ >>> name += 'Yvonne' ↵ >>> name += ' ' ↵ >>> name += 'Smith' ↵ >>> print(name) ↵ Kelly Yvonne Smith >>></pre>	<pre># name is 'Kelly' # name is 'Kelly ' # name is 'Kelly Yvonne' # name is 'Kelly Yvonne ' # name is 'Kelly Yvonne Smith'</pre>
--	---

Strings Are Immutable

- **Strings are immutable**
 - Once they are created, they cannot be changed
 - Concatenation doesn't actually change the existing string, but rather creates a new string and assigns the new string to the previously used variable
 - Cannot use an expression of the form
 - *string[index] = new_character*
 - Statement of this type will raise an exception

Strings Are Immutable (cont'd.)

- Because strings are immutable, you cannot use an expression in the form *string[index]* on the left side of an assignment operator. For example, the following code will cause an error:

Assign 'Bill' to friend.

```
friend = 'Bill'
```

Can we change the first character to 'J'?

```
friend[0] = 'J' # No, this will cause an error!
```

- The last statement in this code will raise an exception because it attempts to change the value of the first character in the string 'Bill'.



String Slicing

- **Slice**: span of items taken from a sequence, known as *substring*
 - Slicing format: `string[start:end:step]`
 - Expression will return a string containing a **copy of the characters from** `start` up to, but not including, `end`
 - If `start` not specified, 0 is used for start index
 - If `end` not specified, `len(string)` is used for end index
 - Slicing expressions can include a step value and negative indexes relative to end of string



String Slicing

- **Examples**

- `full_name = 'Patty Lynn Smith'`
- `middle_name = full_name[6:10]` ➡ `middle_name= 'Lynn'`
- `first_name = full_name[:5]` ➡ `first_name= 'Patty'`
- `last_name = full_name[11:]` ➡ `last_name= 'Smith'`
- `last_name = full_name[-5:]` ➡ `last_name= 'Smith'`
- `last_name = full_name[-1:-3:-1]` ➡ `last_name= 'ht'`

Testing, Searching, and Manipulating Strings

- You can use the `in` operator to determine whether one string is contained in another
 - General format: `string1 in string2`
 - `string1` and `string2` can be string literals or variables referencing strings

```
text = 'Four score and seven years ago'
```

```
if 'seven' in text:
```

```
    print('The string "seven" was found.')
```



string "seven" was found.

```
else:
```

```
    print('The string "seven" was not found.')
```


Testing, Searching, and Manipulating Strings

- Similarly, you can use the `not in` operator to determine whether one string is not contained in another string

```
names = 'Bill Joanne Susan Chris Juan Katie'
```

```
if 'Pierre' not in names:
```

```
    print('Pierre was not found.')
```

```
else:
```

```
    print('Pierre was found.')
```



Pierre was not found.

String Methods

- **Strings in Python have many types of methods, divided into different types of operations**
 - General format:
mystring.method(arguments)
- **Some methods test a string for specific characteristics**
 - Generally Boolean methods, that return `True` if a condition exists, and `False` otherwise

String Methods (cont'd.)

Table 8-1 Some string testing methods

Method	Description
<code>isalnum()</code>	Returns true if the string contains only alphabetic letters or digits and is at least one character in length. Returns false otherwise.
<code>isalpha()</code>	Returns true if the string contains only alphabetic letters and is at least one character in length. Returns false otherwise.
<code>isdigit()</code>	Returns true if the string contains only numeric digits and is at least one character in length. Returns false otherwise.
<code>islower()</code>	Returns true if all of the alphabetic letters in the string are lowercase, and the string contains at least one alphabetic letter. Returns false otherwise.
<code>isspace()</code>	Returns true if the string contains only whitespace characters and is at least one character in length. Returns false otherwise. (Whitespace characters are spaces, newlines (<code>\n</code>), and tabs (<code>\t</code>).
<code>isupper()</code>	Returns true if all of the alphabetic letters in the string are uppercase, and the string contains at least one alphabetic letter. Returns false otherwise.

String Methods (cont'd.)

```
string1 = '1200'  
if string1.isdigit():  
    print(string1, 'contains only digits.')  
else:  
    print(string1, 'contains characters other than digits.')
```



```
string1 = '1200'  
string1.isdigit()
```



True



1200 contains only digits.

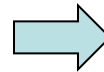
```
string2 = '123abc'  
if string2.isdigit():  
    print(string2, 'contains only digits.')  
else:  
    print(string2, 'contains characters other than digits.')
```



123abc contains characters other than digits.

Program 8-5 (string_test.py)

```
1 # This program demonstrates several string testing methods.
2
3 def main():
4     # Get a string from the user.
5     user_string = input('Enter a string: ')
6
7     print('This is what I found about that string:')
8
9     # Test the string.
10    if user_string.isalnum():
11        print('The string is alphanumeric.')
12    if user_string.isdigit():
13        print('The string contains only digits.')
14    if user_string.isalpha():
15        print('The string contains only alphabetic characters.')
16    if user_string.isspace():
17        print('The string contains only whitespace characters.')
18    if user_string.islower():
19        print('The letters in the string are all lowercase.')
20    if user_string.isupper():
21        print('The letters in the string are all uppercase.')
22
23 # Call the string.
24 main()
```



Program Output (with input shown in bold)

Enter a string: **abc** ↵

This is what I found about that string:

The string is alphanumeric.

The string contains only alphabetic characters.

The letters in the string are all lowercase.

Program Output (with input shown in bold)

Enter a string: **123** ↵

This is what I found about that string:

The string is alphanumeric.

The string contains only digits.

Program Output (with input shown in bold)

Enter a string: **123ABC** ↵

This is what I found about that string:

The string is alphanumeric.

The letters in the string are all uppercase.



string_test.py

String Methods (cont'd.)

- **Some methods return a copy of the string, to which modifications have been made**
 - Simulate strings as mutable objects
- **String comparisons are case-sensitive**
 - Uppercase characters are distinguished from lowercase characters
 - `lower` and `upper` methods can be used for making case-insensitive string comparisons

String Methods (cont'd.)

Table 8-2 String Modification Methods

Method	Description
<code>lower()</code>	Returns a copy of the string with all alphabetic letters converted to lowercase. Any character that is already lowercase, or is not an alphabetic letter, is unchanged.
<code>lstrip()</code>	Returns a copy of the string with all leading whitespace characters removed. Leading whitespace characters are spaces, newlines (<code>\n</code>), and tabs (<code>\t</code>) that appear at the beginning of the string.
<code>lstrip(char)</code>	The <i>char</i> argument is a string containing a character. Returns a copy of the string with all instances of <i>char</i> that appear at the beginning of the string removed.
<code>rstrip()</code>	Returns a copy of the string with all trailing whitespace characters removed. Trailing whitespace characters are spaces, newlines (<code>\n</code>), and tabs (<code>\t</code>) that appear at the end of the string.
<code>rstrip(char)</code>	The <i>char</i> argument is a string containing a character. The method returns a copy of the string with all instances of <i>char</i> that appear at the end of the string removed.
<code>strip()</code>	Returns a copy of the string with all leading and trailing whitespace characters removed.
<code>strip(char)</code>	Returns a copy of the string with all instances of <i>char</i> that appear at the beginning and the end of the string removed.
<code>upper()</code>	Returns a copy of the string with all alphabetic letters converted to uppercase. Any character that is already uppercase, or is not an alphabetic letter, is unchanged.

String Methods (cont'd.)

```
letters = 'WXYZ'
```

```
print(letters, letters.lower())
```



WXYZ wxyz

```
letters = 'abcd'
```

```
print(letters, letters.upper())
```



abcd ABCD

```
letters = 'abcd9999'
```

```
print(letters, letters.rstrip('9'))
```



abcd9999 abcd

String Methods (cont'd.)

```
letters = ' _WXYZ _' ↵  
print(letters, letters.strip(), '!') ↵
```



```
_WXYZ _WXYZ_!
```

```
>>> letters = '\tabcd' ↵  
>>> letters ↵  
'\tabcd'
```



```
>>> print(letters, letters.lstrip()) ↵  
abcd abcd
```

```
letters = '89abcd889' ↵  
print(letters, letters.strip('89')) ↵
```



```
89abcd889 abcd
```

```
'www.mama.mta.ac.il'.strip('c.liow') ↵
```



```
'mama.mta.a'
```

String Methods (cont'd.)

Table 8-3 Search and replace methods

Method	Description
<code>endswith(<i>substring</i>)</code>	The <i>substring</i> argument is a string. The method returns true if the string ends with <i>substring</i> .
<code>find(<i>substring</i>)</code>	The <i>substring</i> argument is a string. The method returns the lowest index in the string where <i>substring</i> is found. If <i>substring</i> is not found, the method returns -1.
<code>replace(<i>old</i>, <i>new</i>)</code>	The <i>old</i> and <i>new</i> arguments are both strings. The method returns a copy of the string with all instances of <i>old</i> replaced by <i>new</i> .
<code>startswith(<i>substring</i>)</code>	The <i>substring</i> argument is a string. The method returns true if the string starts with <i>substring</i> .

Find(value, start, stop)

Value Required. The value to search for

Start Optional. Where to start the search. Default is 0

End Optional. Where to end the search. Default is to the end of the string

String Methods (cont'd.)

```
filename = input('Enter the filename: ')
if filename.endswith('.txt'):
    print('That is the name of a text file.')
elif filename.endswith('.py'):
    print('That is the name of a Python source file.')
elif filename.endswith('.doc'):
    print('That is the name of a word processing
document.')
else:
    print('Unknown file type.')
```

String Methods (cont'd.)

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

```
string = 'Four score and seven years ago'
position = string.find('seven')
if position != -1:
    print('The word "seven" was found at index', position)
else:
    print('The word "seven" was not found.')
```



The word "seven" was found at index 15

```
string = 'Four score and seven years ago'
new_string = string.replace('years', 'days')
print(new_string)
```



Four score and seven days ago

The Repetition Operator

- **Repetition operator: makes multiple copies of a string and joins them together**
 - The * symbol is a repetition operator when applied to a string and an integer
 - String is left operand; number is right
 - General format: *string_to_copy* * *n*
 - Variable references a new string which contains multiple copies of the original string

The Repetition Operator

```
my_string = 'w' * 5
```

```
Print(my_string) ↵
```

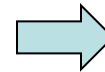
```
WWWWW
```

```
print('Hello' * 5) ↵
```

```
HelloHelloHelloHelloHello
```

Program 8-8 (repetition_operator.py)

```
1 # This program demonstrates the repetition operator.
2
3 def main():
4     # Print nine rows increasing in length.
5     for count in range(1, 10):
6         print('Z' * count)
7
8     # Print nine rows decreasing in length.
9     for count in range(8, 0, -1):
10        print('Z' * count)
11
12 # Call the main function.
13 main()
```



Program Output

```
Z
ZZ
ZZZ
ZZZZ
ZZZZZ
ZZZZZZ
ZZZZZZZ
ZZZZZZZZ
ZZZZZZZZZ
ZZZZZZZZZ
ZZZZZZZZZ
ZZZZZZZZ
ZZZZZZZ
ZZZZZZ
ZZZZZ
ZZZZ
ZZZ
ZZ
Z
```



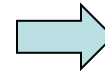
repetition_operator.py

Splitting a String

- **split method: returns a list containing the words in the string**
 - By default, uses space as separator
 - Can specify a different separator by passing it as an argument to the `split` method

Program 8-9 (string_split.py)

```
1 # This program demonstrates the split method.
2
3 def main():
4 # Create a string with multiple words.
5 my_string = 'One two three four'
6
7 # Split the string.
8 word_list = my_string.split()
9
10 # Print the list of words.
11 print(word_list)
12
13 # Call the main function.
14 main()
```



Program Output

['One', 'two', 'three', 'four']

```
date_string = '11/26/2018'
date_list = date_string.split('/')

```

After this statement executes, the date_list variable will reference this list:
['11', '26', '2018']



Program 8-9 (string_split.py)

```
1 # This program demonstrates the split method.
```

```
2
```

```
3 def main():
```

```
4 # Create a string with multiple words.
```

```
5 my_string = 'One two three four'
```

```
6
```

```
7 # Split the string.
```

```
8 word_list = my_string.split()
```

```
9
```

```
10 # Print the list of words.
```

```
11 print(word_list)
```

```
12
```

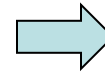
```
13 # Call the main function.
```

```
14 main()
```

```
str = "a***b*c"
```

```
lst = str.split("***")
```

```
["a",,, "b", "c"]
```



Program Output

```
['One', 'two', 'three', 'four']
```

```
str = "a  b c"
```

```
lst = str.split()
```

```
["a", "b", "c"]
```

```
str = "a  b c"
```

```
lst = str.split(" ")
```

```
["a",,, "b", "c"]
```

```
date_string = '11/26/2018'
```

```
date_list = date_string.split('/')
```

After this statement executes, the date_list variable will reference this list:

```
['11', '26', '2018']
```

